

Overview & HLD

CRDT deep dive

Tech choices

Project structure

Workflows

Dev phases

WHAT YOU'RE BUILDING

CollabCode — Google Docs for code

Multiple users edit the same file simultaneously from different browsers. Edits appear in real-time for all users, cursors are tracked per user, conflicts are resolved automatically without any locking, and the document always converges to the same state. Syntax highlighting, language selection, and session history included.

THE CORE PROBLEM — WHY THIS IS HARD

Problem Concurrent edits break naively

User A and User B both start with "Hello". A inserts "World" at position 5. B deletes "Hello" at position 0. If you apply both edits sequentially using raw character positions, one edit corrupts the other. This is called the **lost update problem** and it's why you can't build this with a simple "send the diff via WebSocket" approach.

Solution CRDTs — conflict-free replicated data types

A CRDT is a data structure designed so that concurrent operations always merge correctly without coordination. You use the **Yjs library** which implements a CRDT called Y.Doc. Instead of tracking character positions (which break under concurrency), Yjs tracks every character as a unique node with a logical clock ID. Merging always works, always produces the same result, regardless of order operations arrive.

HLD — 4 LAYERS

Layer 1 Client — React + Monaco Editor + Yjs

Monaco is the same editor VS Code uses. Yjs binds to it via `y-monaco` so that every keystroke becomes a CRDT operation, not a raw string diff. Users see each other's cursors as coloured carets. The client connects to the server via WebSocket (`y-websocket-provider`) and optionally persists to IndexedDB (`y-indexeddb`) so the doc loads offline too.

Layer 2 Sync server — Node.js + y-websocket

A thin WebSocket server that receives CRDT update messages from any client and broadcasts them to all other clients in the same room. It doesn't understand the document contents — it just relays binary Yjs update blobs. Optionally persists the latest doc state to Redis so new joiners don't have to replay all history.

Layer 3 Persistence — PostgreSQL + Redis

PostgreSQL stores room metadata, user accounts, and session history. Redis stores the latest serialised Y.Doc state per room (binary blob, updated on every change) so new users joining an existing session get the current state instantly without replaying all operations from the beginning.

Layer 4 Awareness layer — cursor + presence

Yjs has a built-in "awareness" protocol on top of WebSocket. Each client broadcasts its cursor position, username, and colour. All other clients receive this and Monaco renders coloured cursor decorations in real-time. This is separate from the document sync — awareness data is ephemeral and lost on disconnect.

THE CONCEPT — WITHOUT THE MATHS

Why character positions fail

Imagine two users. Both see: Hello. User A inserts 'l' at index 2 → Hello. User B inserts 'l' at index 4 → Hello!. If A's update arrives first and shifts characters right, B's index 4 now points to 'o' instead of end. B's 'l' lands in the wrong place. Every simple diff-based approach has this flaw.

How Yjs fixes it — unique IDs, not positions

Instead of "insert at position 5", Yjs says "insert after the node with ID (clientA, clock:7)". IDs are stable — they don't shift when other characters are inserted. A character always knows what it comes after, regardless of what happened concurrently. When two clients insert at the same position simultaneously, a deterministic tiebreaker (client ID) decides order. Same inputs always produce the same result on every client.

CRDT VS OT — THE QUESTION INTERVIEWERS LOVE

Operational transformation (OT) — the older approach

Google Docs used OT. The idea: transform an operation based on what happened concurrently. "insert at 5" becomes "insert at 6" if someone inserted before position 5 first. Works, but requires a central server to order all operations. Hard to implement correctly, especially for rich text. Google has a team dedicated to it.

CRDTs — the modern approach (what you're using)

No central ordering needed. Each client applies operations independently and the state always converges. Yjs's algorithm (called YATA) is peer-to-peer friendly — works even offline. Slightly higher memory use than OT, but this is negligible for code files. Figma, Linear, and most modern collaborative tools use CRDTs.

YJS KEY CONCEPTS YOU MUST KNOW

- 1 Y.Doc**
The root shared document. Every client has one. They sync by exchanging binary update messages.
- 2 Y.Text**
A shared text type inside a Y.Doc. Monaco's y-monaco binding listens to this and applies changes. All concurrent edits go through Y.Text — never touch Monaco's model directly.
- 3 Provider**
How the Y.Doc syncs with others. y-websocket provider connects to your server and sends/receives update blobs. y-indexeddb provider persists to the browser so the doc survives page refresh offline.
- 4 Awareness**
Yjs's ephemeral state layer. Use it for cursor positions, usernames, online status. Awareness data is NOT part of the document and NOT persisted — it's just broadcast to current peers.

```
import * as Y from 'yjs'
import { WebsocketProvider } from 'y-websocket'
import { MonacoBinding } from 'y-monaco'

const ydoc = new Y.Doc()
const ytext = ydoc.getText('code') // shared text

const provider = new WebsocketProvider(
  'ws://localhost:1234', 'room-abc', ydoc
)

// Bind to Monaco editor
const binding = new MonacoBinding(
  ytext, editor.getModel(), new Set([editor]), provider.awareness
)

// Set your cursor colour + name
provider.awareness.setLocalStateField('user', {
  name: 'Arjun', color: '#F08080'
})
```

Show me the full code ↗

Overview & HLD

CRDT deep dive

Tech choices

Project structure

Workflows

Dev phases

EVERY TECHNOLOGY AND WHY



React + Vite

Frontend framework

You know it. Component model maps cleanly to editor, sidebar, and toolbar.



Monaco Editor

Code editor

Same engine as VS Code. Built-in syntax highlighting for 50+ languages, TypeScript IntelliSense, themes. Massive wow factor — looks professional instantly.



Yjs + y-monaco

CRDT engine

The industry standard CRDT library. y-monaco is the official Monaco binding. Used by Notion, Gitpod, and dozens of editors.



y-websocket server

Sync server

Official Yjs WebSocket server. Handles rooms, update relaying, and optional persistence. Saves you writing the sync logic yourself.



Node.js + Express

REST API

Auth, room management, session history. You already know it.



Redis (Upstash)

Doc state cache

Stores the serialised Y.Doc binary per room. New users joining get current state instantly — no full history replay.



PostgreSQL (Supabase)

Persistence

Rooms, users, session snapshots, history. Free tier is plenty for a portfolio project.



Docker Compose

Local dev

Runs y-websocket server + Redis together locally. Same config deploys to VPS.



@monaco-editor/react

Monaco wrapper

Official React wrapper. Handles Monaco's complicated loading lifecycle so you don't have to.

FOLDER STRUCTURE

```
collab-editor/ └─ client/ # React + Vite frontend | └─ src/ | | └─ components/ | | | └─ Editor.jsx #
Monaco + Yjs binding | | | └─ CollabCursors.jsx # coloured cursor decorations | | | └─ Sidebar.jsx #
online users list | | | └─ Toolbar.jsx # Language select, share button | | | └─ HistoryPanel.jsx #
past session snapshots | | └─ hooks/ | | | └─ useYjs.js # Y.Doc + provider setup | | | └─
useAwareness.js # cursor + presence | | | └─ useMonaco.js # Monaco instance ref | | └─ pages/ | | |
└─ Home.jsx # create / join room | | | └─ Room.jsx # main editor view | | └─ App.jsx | └─
package.json | └─ server/ # Node.js REST API | └─ routes/ | | └─ auth.js # register, login, JWT | |
└─ rooms.js # create room, get history | └─ db.js # PostgreSQL connection | └─ redis.js # Redis
client | └─ index.js | └─ sync-server/ # y-websocket sync server | └─ server.js # custom y-websocket
setup | └─ persistence.js # save/load Y.Doc to Redis | └─ docker-compose.yml └─ .env
```

DATABASE SCHEMA

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  color TEXT DEFAULT '#F08040' -- cursor colour
);

CREATE TABLE rooms (
  id TEXT PRIMARY KEY,          -- short nanoid, used as room URL
  name TEXT,
  language TEXT DEFAULT 'javascript',
  owner_id UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE snapshots (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id TEXT REFERENCES rooms(id),
  content TEXT,                 -- full code text at that moment
  saved_at TIMESTAMPTZ DEFAULT now(),
  saved_by UUID REFERENCES users(id)
);
```



WORKFLOW 1 — USER JOINS A ROOM AND GETS CURRENT DOC STATE

- 1 User opens /room/abc-123**
React renders Room.jsx. useYjs hook fires — creates a new Y.Doc and connects a WebSocketProvider to ws://sync-server/abc-123.
- 2 Sync server checks Redis for existing state**
If a Y.Doc state blob exists for room abc-123 in Redis, the server sends it immediately as a "sync step 1" message. The client applies it and the editor fills with the current document content instantly.
- 3 MonacoBinding attaches**
Once the Y.Doc has initial state, MonacoBinding links Y.Text to Monaco's model. From this point, every keystroke in Monaco creates a Y.Text operation, and every remote operation updates Monaco.
- 4 Awareness broadcast**
Client sets its cursor state (username, colour, cursor position). All other connected clients receive it and render a coloured cursor decoration in Monaco at the correct position.

WORKFLOW 2 — CONCURRENT EDIT (THE CORE CRDT FLOW)

1 User A types 'x' at position 3

y-monaco intercepts the Monaco change event. Creates a YText insert operation with a unique ID (clientA, clock:15). Applies it locally to the Y.Doc immediately — no waiting. Monaco shows 'x' instantly.

2 Y.Doc serialises the update

Yjs encodes the operation as a compact binary blob (typically 20–50 bytes for a single character). The WebSocketProvider sends this blob to the sync server.

3 Sync server broadcasts to all room members

The sync server receives the binary blob and forwards it to every other WebSocket connection in room abc-123. It does not parse the blob — just relays bytes.

4 User B's Y.Doc applies the remote update

Yjs merges the incoming operation using CRDT rules. If User B also typed at the same position simultaneously, Yjs resolves the conflict deterministically. Both documents end up identical. Monaco re-renders.

5 Redis updated

Sync server serialises the full Y.Doc state and writes it to Redis under key room:abc-123. New joiners get this state immediately.

WORKFLOW 3 — SESSION SNAPSHOT (HISTORY FEATURE)

1 User clicks "Save snapshot"

Client calls `POST /api/rooms/:id/snapshots` with the current plain text content (`ytext.toString()`).

2 Server saves to PostgreSQL

The snapshot row is inserted with the full code text, timestamp, and user ID. Snapshots are read-only historical records — you can view them but not restore destructively.

3 History panel shows all snapshots

`GET /api/rooms/:id/snapshots` returns all saved snapshots. User can click any to view the code at that point in a read-only Monaco instance side-by-side with the live editor.

DEVELOPMENT EXECUTION — CLICK EACH PHASE

1 Setup + Monaco editor working

Day 1-2

- > Create React app with Vite, install Monaco
- > Get Monaco rendering with JS syntax highlighting — this alone looks impressive
- > Add language selector dropdown (javascript, python, go, rust etc.)
- > Add VS Code dark theme

```
npm install @monaco-editor/react
npm install yjs y-websocket y-monaco y-indexeddb
```

```
// Editor.jsx — Monaco renders, no collab yet
import Editor from '@monaco-editor/react'

export default function CodeEditor() {
  return (
    <Editor
      height="100vh"
      defaultLanguage="javascript"
      theme="vs-dark"
      options={{ fontSize: 14, minimap: { enabled: false } }}
    />
  )
}
```

Goal: a beautiful working code editor in the browser. Screenshot this for your resume.



CRDT deep dive Tech choices Project structure Workflows Dev phases Interview prep

DEVELOPMENT EXECUTION — CLICK EACH PHASE

1 Setup + Monaco editor working

Day 1-2

2 Auth + room creation

Day 3-4

3 Yjs + y-websocket sync server

Week 1, Day 5-7

4 Coloured cursors + presence (awareness)

Week 2, Day 1-3

5 Redis persistence + new joiner state

Week 2, Day 4-5

6 History snapshots + extra features

Week 2, Day 6-7

7 Deploy — Vercel + Railway/Render

Week 3

2

Auth + room creation

Day 3-4

- Node.js + Express: POST /auth/register, POST /auth/login (JWT)
- POST /rooms — generates a short room ID (use nanoid), saves to PG
- Home page: "Create room" button → generates ID → navigate to /room/:id
- "Join room" — enter room ID or paste shareable link
- Store JWT in localStorage, pass as query param to WebSocket later

```
npm install nanoid bcryptjs jsonwebtoken
```

- ☐ Set up y-websocket server as a separate Node process (port 1234)
- 🔗 Connect WebSocketProvider in the frontend to ws://localhost:1234/room-id
- 🔗 Bind Y.Text to Monaco model using MonacoBinding
- ✓ Test: open two browser tabs to the same room URL, type in one — appears in the other

```
// sync-server/server.js
const { WebSocketServer } = require('y-websocket/bin/utils')
const http = require('http')

const server = http.createServer()
const wss = new WebSocketServer({ server })
server.listen(1234, () => console.log('Sync server on :1234'))
```

```
// client: hooks/useYjs.js
import * as Y from 'yjs'
import { WebSocketProvider } from 'y-websocket'
import { IndexeddbPersistence } from 'y-indexeddb'

export function useYjs(roomId) {
  const ydoc = useMemo(() => new Y.Doc(), [])
  const ytext = useMemo(() => ydoc.getText('code'), [ydoc])

  useEffect(() => {
    const wsProvider = new WebSocketProvider(
      'ws://localhost:1234', roomId, ydoc
    )
    const idbProvider = new IndexeddbPersistence(roomId, ydoc)
    return () => { wsProvider.destroy(); idbProvider.destroy() }
  }, [roomId, ydoc])

  return { ydoc, ytext }
}
```

- ⌘ Set awareness state with username, colour, cursor position on every Monaco cursor move
- ⌘ Subscribe to awareness changes — render Monaco cursor decorations for each remote user
- 80 Sidebar: show online users list with their assigned colours and usernames
- 80 Show user initials badge at their cursor position in the editor
- 80
- ✓ Test: three tabs open, move cursor in each — other tabs show coloured carets
- 80

```
// useAwareness.js
provider.awareness.setLocalStateField('user', {
  name: currentUser.username,
  color: currentUser.color,
})

provider.awareness.on('change', () => {
  const states = Array.from(provider.awareness.getStates().entries())
  // map to Monaco editor.deltaDecorations() calls
})
```

- ✧ On every Y.Doc update: serialise with `Y.encodeStateAsUpdate(doc)` and SET in Redis
- ✧ On new WebSocket connection: GET from Redis and send as initial sync message
- ✓ Test: type in room, close all tabs, reopen — document is restored from Redis

```
// sync-server/persistence.js
const { encodeStateAsUpdate, applyUpdate } = require('yjs')

async function saveDoc(roomId, ydoc) {
  const state = Buffer.from(encodeStateAsUpdate(ydoc))
  await redis.set(`room:${roomId}`, state)
}

async function loadDoc(roomId, ydoc) {
  const state = await redis.getBuffer(`room:${roomId}`)
  if (state) applyUpdate(ydoc, state)
}
```

6

History snapshots + extra features

Week 2, Day 6-7

- ✎ `POST /rooms/:id/snapshots` — save `ytext.toString()` to PostgreSQL
- 📖 History panel — list snapshots, click to view in read-only Monaco
- 📖 Share button — copy room URL to clipboard, show QR code
- ✎ Download button — download current code as a `.js` / `.py` / `.go` file
- 📖 Run code (optional) — send to a sandboxed Judge0 API, show output panel

[Add code execution ↗](#)

7

Deploy — Vercel + Railway/Render

Week 3

- ☁ Deploy React frontend to Vercel (free, automatic from GitHub)
- ☁ Deploy REST API to Render.com free tier
- ☁ Deploy sync server to Railway.app (supports WebSockets, free tier available)
- ⚡ Upstash Redis for persistence (free tier)
- 🗄 Supabase PostgreSQL (free tier)
- ✓ Total cost: ₹0